

`</> Thread.start()`

RustyJDK · Sistemas de bajo nivel

Concurrencia e hilos en una JVM en Rust

Informe técnico — qué hay y qué se podría hacer

Autor: **Esteban Nicolás Larena**

Fecha: **26 de junio de 2026**

Contenido

1 · Resumen ejecutivo	3
2 · Estado actual (lo que ya corre)	3
3 · El rascacielos de la concurrencia	4
4 · Estado verificado contra el código	5
5 · Los dos cimientos: park/unpark y CAS	6
6 · Ruta crítica — GIL primero (GIL $\neq$ JIT)	6
7 · Plan por hitos (H0–H6)	7
8 · Consideraciones de Rust y arquitectura	9
9 · Riesgos y orden recomendado	10

1 · Resumen ejecutivo

Este informe es un **deep-dive del Hito A6 (concurrency)** de RustyJDK: separa, contra el código real, **lo que ya funciona** de **lo que se podría hacer** para llegar a un *multithread profesional*. La meta no es el cómputo numérico del LLM —que vive en **kernels off-heap** (rayon/CUDA)— sino la **corrección** de la concurrency Java sobre el heap administrado.

Hoy la VM ejecuta **green threads cooperativos** sobre un único hilo de SO, con **monitores** (`synchronized` de bloques y métodos), señalización `wait` / `notify` y `join` / `sleep`. Es la *planta baja* de un edificio cuya cumbre es `java.util.concurrent`.

*Tesis del informe: el salto grande **no** es el JIT ni terminar el monitor, sino **cambiar el sustrato** de green threads a hilos de SO. El camino canónico es **GIL primero** (correcto pero sin paralelismo) y después **sacar el GIL**. El JIT es ortogonal (Fase F3) y no está en esta ruta.*

Horizontes: Base el núcleo Avanzado segunda pasada Cumbre lo más difícil

2 · Estado actual (lo que ya corre)

Verificado en el intérprete (`bytecode_interpreter.rs` + los `invoke*`) y con tests verdes:

- **Green threads (Fase 0).** `GreenThread` = un call stack propio; scheduler **round-robin cooperativo** que conmuta en el *safe point* (entre opcodes); el stack activo vive en `self.frames` y se intercambia con el del hilo entrante. `Thread.start()` spawna (dispatch virtual de `run()`); `join` y `sleep` (reloj lógico de opcodes). El GC toma las raíces de **todos** los hilos (`parked()`). Verificado: dos workers + `main` → `100`.
- **Monitores.** `Monitor` = dueño + conteo reentrante + *blocked-set* + *wait-set*. `synchronized` en sus dos formas: **bloques** (`monitorenter` / `monitorexit`) y **métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma/suelta el monitor del receptor, o del `Class` en `static synchronized`, en un único punto de *release* imposible de saltar, también en el *unwind* de excepción).
- **Señalización.** `wait` / `notify` / `notifyAll`: libera el monitor guardando el conteo de reentrada, parquea en el wait-set y lo re-adquiere al despertar. Verificado: exclusión mutua real (2×100 → 200 vs control sin lock que pierde updates) y handshake productor/consumidor (`wait` → `notify` → 42).
- **`IllegalMonitorStateException`** (en el working tree, sin commitear): gate `owns_monitor` sobre `monitorexit` / `wait` / `notify` (JVMS §6.5 / JLS 17.2).
- **El seam.** El heap está tras `HeapService`, su único dueño; toda escritura de referencia pasa por `store_reference` (write barrier no-bypassable) — **el punto donde irá la sincronización** cuando los hilos pasen a ser de SO.

3 · El rascacielos de la concurrencia

Cada planta se apoya en la de abajo: no hay `ReentrantLock` sin CAS, ni CAS útil sin un modelo de memoria, ni modelo de memoria que *importe* sin hilos reales que lo hagan necesario.

RASCACIELOS DE LA CONCURRENCIA · construir hacia arriba ↑	
[6] java.util.concurrent · lo que usa la gente AQS · ReentrantLock · Condition · Semaphore · Latch BlockingQueue · ConcurrentHashMap · Executor / pools	 FALTA
[5] Atómicos / CAS · raíz lock-free compareAndSwap · AtomicInteger / AtomicLong / Reference	 FALTA
[4] Modelo de Memoria (JMM) · visibilidad + orden volatile · happens-before · fences · final · no-tearing	 FALTA
[3] API de Thread · control de hilos  start · run · join · sleep  interrupt · yield · currentThread · prioridad · daemon	 PARC.
[2] Monitor intrínseco · ← ACÁ ESTAMOS  monitorenter/exit · synchronized (bloques Y métodos)  reentrancia · wait / notify / notifyAll · IMSE*  wait(timeout) · interrupts · GC-safe (hoy keyed-by-offset)	 PARC.
[1] Substrato de ejecución · paralelismo real  green threads (cooperativo, 1 carrier OS)  park/unpark · GIL · hilos de SO sin GIL · stop-the-world	 PARC.
▲ todo descansa en la planta 1 ▲	
* IMSE = IllegalMonitorStateException (en el working tree)	

Estado:  hecho  parcial  falta

4 · Estado verificado contra el código

Planta por planta, lo que falta para «profesional» (revisado en `bytecode_interpreter.rs`, `invoke*.rs` y el `Thread.java` de `bootstrap/`):

Planta	Estado	Qué falta
6 · <code>java.util.concurrent</code>	 falta	AQS (<code>AbstractQueuedSynchronizer</code> , base de casi todo), <code>ReentrantLock</code> / <code>ReadWriteLock</code> , <code>Condition</code> , <code>Semaphore</code> , <code>CountDownLatch</code> , <code>CyclicBarrier</code> , <code>BlockingQueue</code> , <code>ConcurrentHashMap</code> , <code>Executor</code> /pools, <code>CompletableFuture</code>
5 · Atómicos / CAS	 falta	<code>compareAndSwap</code> (la instrucción atómica raíz) y los <code>Atomic*</code> . Sin esto no hay lock-free ni AQS
4 · Modelo de Memoria (JMM)	 falta	<code>volatile</code> , happens-before, barreras/fences, semántica de <code>final</code> , no-tearing de <code>long</code> / <code>double</code>
3 · API de <code>Thread</code>	 parcial	Hecho: <code>start</code> / <code>run</code> / <code>join()</code> / <code>sleep(long)</code> . Falta: <code>interrupt</code> / <code>InterruptedException</code> , <code>yield</code> , <code>currentThread</code> , prioridad, daemon + shutdown, nombre/id, <code>getState</code> , <code>isAlive</code> , patrón <code>Runnable</code> , <code>start</code> -dos-veces, <code>UncaughtExceptionHandler</code> , <code>ThreadLocal</code>
2 · Monitor intrínseco	 parcial	Hecho: <code>monitorenter</code> / <code>exit</code> , <code>synchronized</code> (bloques y métodos, incl. <code>static</code>), reentrancia, <code>wait</code> / <code>notify</code> / <code>notifyAll</code> , IMSE*. Falta: <code>wait(timeout)</code> , interrupciones del wait, monitor <i>GC-safe</i> (hoy se keyea por <i>offset</i> del heap; al compactar, cambia), biased/thin locks, detección de deadlock
1 · Substrato de ejecución	 parcial	Hecho: green threads (scheduler cooperativo, 1 carrier). Falta: <code>park</code> / <code>unpark</code> , GIL, hilos de SO sin GIL (paralelismo real), safe-points stop-the-world, preempción

5 · Los dos cimientos: park/unpark y CAS

De todo el edificio, **dos primitivas** sostienen el resto: **park / unpark** (planta 1) y **CAS** (planta 5). De esas dos se deduce *casi todo* lo de arriba — AQS, los locks, los atómicos, los pools.

- **park / unpark** — bloquear/despertar un hilo sin *busy-wait*. Hoy el scheduler cede cada opcode (no hace falta), pero con hilos de SO es *la* primitiva de bloqueo. Sobre ella se construyen **LockSupport** → AQS → todos los locks.
- **CAS** (**compareAndSwap**) — la operación atómica raíz. Sin ella no hay estructuras lock-free ni AQS. En Rust se mapea directo a **std::sync::atomic** (**compare_exchange**).

6 · Ruta crítica — GIL primero (GIL ≠ JIT)

```

HOY green threads → cerrar monitor IMSE✓ · wait(t) · interrupt → cambiar substrato OS threads + GIL → sacar el GIL (park/unpark reales) → desbloquear "pro" JMM → CAS → AQS → java.util.concurrent
  
```

El camino canónico —el de CPython y las JVM tempranas— es **GIL primero**: hilos de SO con un *lock global* que serializa el intérprete (simple y correcto, pero todavía **sin paralelismo real**) y después **sacar el GIL** (locks finos por estructura + TLABs + GC stop-the-world). Recién ahí **volatile** y los *fences* dejan de ser decorativos: con green threads no hay reordenamientos reales que tapar; con hilos de SO, omitirlos rompe el código de forma no-determinista.

Aclaración importante (GIL ≠ JIT). El **GIL** es un lock que serializa el intérprete; habilita hilos de SO correctos sin reescribir la VM. El **JIT** compila bytecode a nativo y vive en la **Fase F3** (track **burst**), fuera de esta ruta. No se necesita JIT para tener hilos de SO ni para un JMM correcto: lo que cuesta es el **refactor de ownership**, no la generación de código nativo.

7 · Plan por hitos (H0–H6)

Cada hito con su criterio de éxito medible. Los dos primeros (H0–H1) se hacen **sobre el sustrato actual** (baratos); H2 es el salto grande.

H0 Cerrar el monitor intrínseco Avanzado

Base Commitear `IllegalMonitorStateException` (gate `owns_monitor`, ya en el working tree)

Avanzado `wait(long)` / `wait(long,int)` con timeout — reusar el reloj de `sleep_until`

Avanzado Interrupción del `wait` → `InterruptedException` (depende de H1)

Cumbre Monitor *GC-safe*: keyar por identidad estable, no por offset (hoy la compactación movería la clave)

✓ **Éxito:** un `wait(t)` vence por tiempo; un monitor sobrevive a un GC compactante sin perder dueño/`wait-set`.

H1 API de `Thread` completa Avanzado

Base `currentThread()`, `yield()`, nombre/id, `isAlive()`, `getState()` (NEW...TERMINATED)

Avanzado `interrupt()` / `isInterrupted()` / `interrupted()` + `InterruptedException` que despierta `wait` / `sleep` / `join`

Avanzado Patrón `new Thread(runnable)` (interfaz `Runnable` + campo `target`); `start` dos veces → `IllegalThreadStateException`

Avanzado Daemon + shutdown cuando solo quedan daemons; prioridad; `UncaughtExceptionHandler`; `ThreadLocal`

✓ **Éxito:** `new Thread(r,"w").start()` corre `r.run()`; `t.interrupt()` saca a un hilo de `sleep` con `InterruptedException`; `getState()` reporta fiel.

H2 Sustrato: hilos de SO + GIL Cumbre

Cumbre Cada `GreenThread` → un `std::thread` real (carrier propio)

Cumbre **GIL:** un lock global del intérprete tomado entre opcodes y cedido en cada safepoint (correcto, todavía serializado)

Cumbre `park` / `unpark` reales sobre el SO (base de `wait` / `join` / contención)

Cumbre Refactor de ownership: heap/monitors/metaspaces compartidos tras el seam

`HeapService.store_reference` (`Arc` + `Send` / `Sync`)

✓ **Éxito:** N hilos de SO ejecutan el mismo programa serializados por el GIL, sin *data races* (mismo resultado que en green threads).

H3 Sacar el GIL — paralelismo real Cumbre

Cumbre Locks finos por estructura (regiones del heap, mapa de monitores, metaspace) en vez del lock único

Cumbre **TLABs** (Thread-Local Allocation Buffers) — alocar sin lock global

Cumbre GC **stop-the-world** con *handshake* de safepoint (todos paran antes de mover objetos)

Cumbre Opcional: biased/thin locks para monitores no contendidos

✓ **Éxito:** dos hilos CPU-bound escalan en paralelo (speedup medible > 1×) sin romper invariantes del GC.

H4 Modelo de memoria (JMM) Cumbre

Cumbre `volatile` read/write con fences (Acquire/Release; SeqCst donde aplique)

Cumbre Happens-before: monitor enter/exit, `volatile`, `start` / `join` (estas dos ya dan aristas), campos `final`

Cumbre No-tearing de `long` / `double`; publicación segura de `final`

✓ **Éxito:** un test de publicación insegura *falla* sin `volatile` y *pasa* con `volatile` bajo hilos de SO.

H5 Atómicos / CAS Cumbre

Cumbre `compareAndSwap` intrínseco (→ `std::sync::atomic::*::compare_exchange`)

Cumbre `AtomicInteger` / `AtomicLong` / `AtomicReference`, `getAndAdd`, un `VarHandle` / `Unsafe` mínimo

✓ **Éxito:** un contador con CAS en bucle desde varios hilos da el total exacto, sin lock.

H6 `java.util.concurrent` Cumbre

Cumbre **AQS** (`AbstractQueuedSynchronizer`) sobre `park` / `unpark` + CAS — la base de casi todo

Cumbre `ReentrantLock` / `ReadWriteLock` / `Condition`, `Semaphore` / `CountDownLatch` / `CyclicBarrier`

Cumbre `BlockingQueue`, `ConcurrentHashMap`, `Executor` / `ThreadPool` / `Future`

✓ **Éxito:** un productor/consumidor con `ArrayBlockingQueue` + `ExecutorService` corre en tu JVM.

8 · Consideraciones de Rust y arquitectura

El costo real del salto de substrato **no es el JIT**: es el **refactor de ownership**. Hoy toda la VM es `&mut self` con un único dueño (heap, monitores, metaspace, frames). Hilos de SO = estado compartido y sincronizado.

- **El seam ya está puesto.** `HeapService.store_reference` es el portón no-bypasseable por donde pasa toda escritura de referencia (hoy aplica el write barrier). Es exactamente donde entrará el lock/atomicidad cuando los hilos sean de SO — no hay que buscar todos los sitios de escritura.
- **Send / Sync**. El estado compartido vive tras `Arc<Mutex<...>>` (o locks por estructura en H3). El **GIL es el primer paso correcto**: un solo `Mutex` grande, fácil de razonar, antes de afinar.
- **El GIL se cede en safepoints**. Ya existe el safepoint entre opcodes (lo usa el GC). Es el mismo punto donde el hilo suelta el GIL y donde el GC stop-the-world hará su *handshake* en H3.
- **Differential / stress testing como oráculo**. El intérprete green-thread es la referencia de corrección: el mismo programa por los dos substratos debe dar el mismo resultado; los tests de estrés (carreras conocidas) cazan los *data races* al sacar el GIL.
- **El cómputo del LLM no entra acá**. El paralelismo pesado vive en kernels off-heap (rayon/CUDA); esta torre es para la *corrección* de la concurrencia Java sobre el heap, no para el número.

9 · Riesgos y orden recomendado

Hito	Riesgo / costo	Nota
H0 · cerrar monitor	Bajo	Autocontenido, sobre el substrato actual. IMSE ya está
H1 · API Thread	Bajo-medio	Mucho es cableado prolijo; <code>interrupt</code> toca el scheduler
H2 · OS threads + GIL	Alto	El refactor de ownership. El salto grande. GIL = correcto pero serializado
H3 · sacar el GIL	Alto	Locks finos + TLABs + GC STW; acá aparecen los <i>data races</i>
H4 · JMM	Medio-alto	Recién útil con H3; fences en accesos <code>volatile</code> y monitor
H5 · CAS	Medio	Mapea directo a <code>std::sync::atomic</code>
H6 · j.u.c	Alto (volumen)	Mucha superficie, pero casi todo se deduce de AQS = park/unpark + CAS

Orden recomendado: H0 → H1 (ganancias baratas sobre el substrato actual) → **H2** (el salto: OS threads + GIL) → H3 (sacar el GIL) → H4 (JMM, ya con paralelismo real) → H5 (CAS) → H6 (`java.util.concurrent`).

Una sola frase: el JIT no habilita los hilos de SO; el refactor de ownership sí. GIL primero, JIT aparte. El monitor está casi listo; el verdadero trabajo empieza en la planta 1.

✓ **Siguiente paso concreto:** committear IMSE, sumar `wait(timeout)` + `interrupt` (H0/H1), y preparar el refactor de `HeapService` para el GIL (H2).